

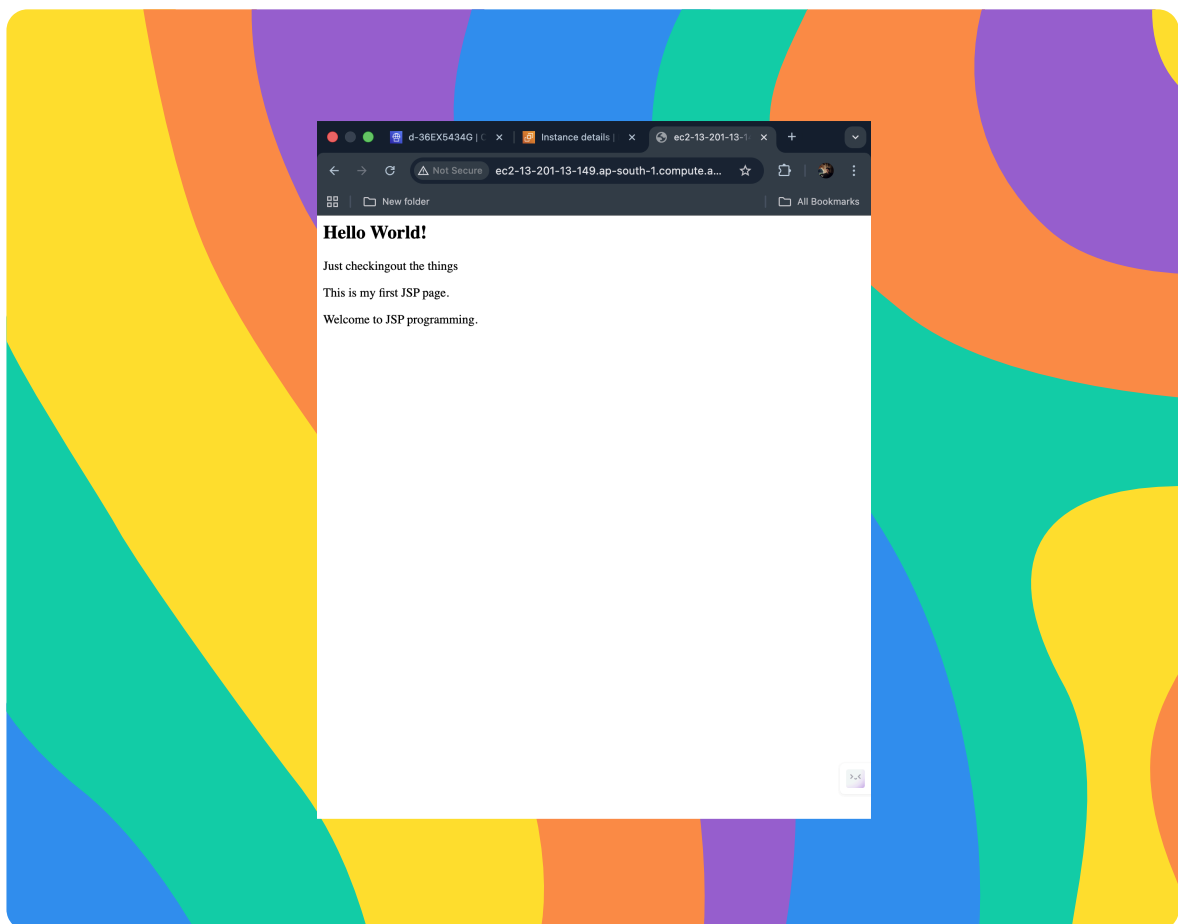


nextwork.org

Deploy a Web App with CodeDeploy



Dhruva Kashyap





Introducing Today's Project!

The goal is to set up AWS CodeDeploy to automate the deployment of the web application to the target environment. This includes configuring the deployment process so application updates are released in a consistent and repeatable way. It also ensures deployments are faster, safer, and easier to roll back if any issue occurs.

Key tools and concepts

In this project, I learned how AWS CodeDeploy automates application deployments to EC2 instances using an `appspec.yml` file and lifecycle event hooks. I also understood how deployment groups use EC2 tags for instance targeting, while IAM roles and the CodeDeploy Agent enable secure communication and execution of deployment scripts. Additionally, I gained practical experience in deployment monitoring, failure troubleshooting through logs, and performing recovery steps when deployments fail.

Project reflection

This project took me approximately 3 hours to complete. It included setting up the deployment infrastructure, preparing deployment scripts and configuration files, and configuring CodeDeploy with the required roles and targeting rules. Time was also spent monitoring deployments and troubleshooting failures to ensure the application was deployed successfully.



Dhruva Kashyap

NextWork Student

nextwork.org

I will work on the next project on the upcoming day as part of continuing the 7 Day DevOps Challenge. It will build on the current setup by adding the next stage of automation into the CI/CD pipeline. This step-by-step progression helps strengthen real-world DevOps workflow understanding.

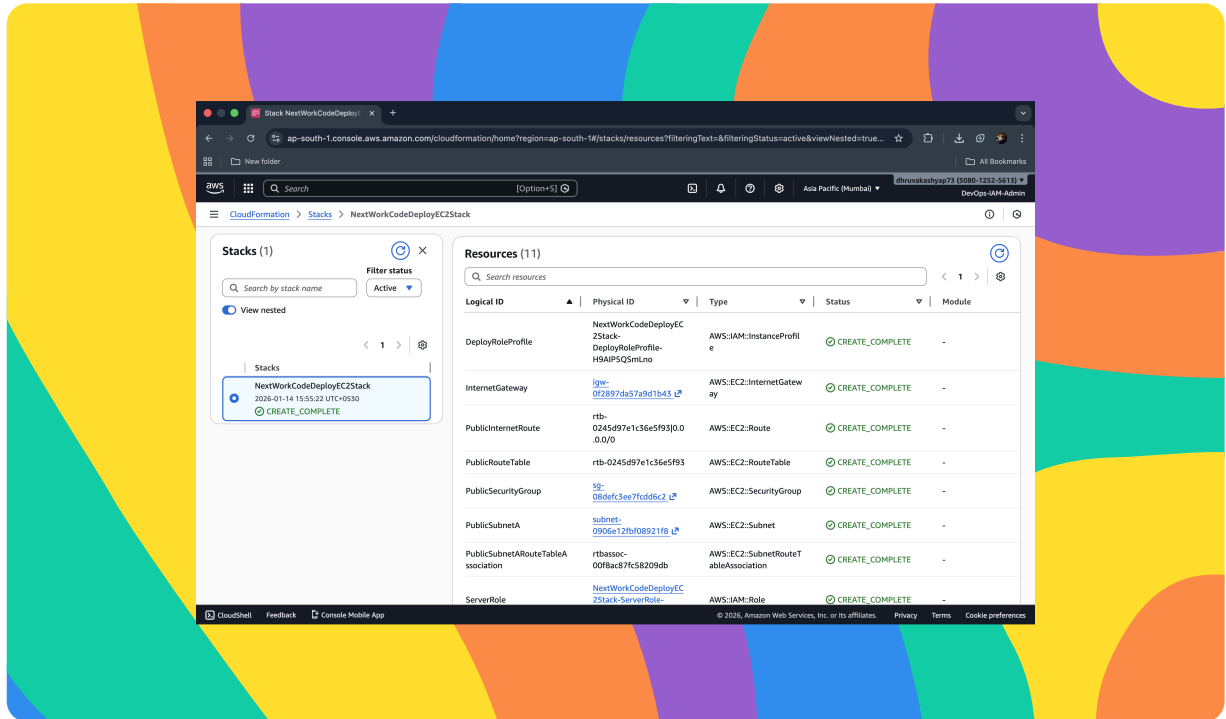


Deployment Environment

An EC2 instance and VPC were launched to create a dedicated deployment environment where the web application can run in a live setup. The VPC provides secure networking components like subnets, routing, and security controls to manage how traffic reaches the instance. This ensures the deployment infrastructure is isolated, consistent, and ready for automated deployments using CodeDeploy.

The EC2 instance and VPC were deployed using AWS CloudFormation. CloudFormation created the required infrastructure automatically from the `nextworkwebapp.yaml` template, ensuring the resources were provisioned consistently and securely.

The CloudFormation template creates both compute and networking resources required to run the application in a deployment environment. This includes a VPC, subnet, route table, internet gateway, and security group to establish secure connectivity and traffic control. It also provisions an EC2 instance inside the network where the web application will be deployed.





Deployment Scripts

Scripts are small automation files that contain a sequence of commands to perform tasks without manual intervention. They help standardize processes like installing software, configuring services, and starting or stopping applications during deployment. In this project, scripts are used to ensure the deployment steps run consistently every time CodeDeploy executes them.

The `install_dependencies.sh` script installs the required server software, including Tomcat and Apache HTTP Server, on the deployment EC2 instance. It also creates an Apache configuration file that forwards incoming web traffic to the Tomcat application running on port 8080. This ensures the environment is properly prepared to host and serve the Java web application.

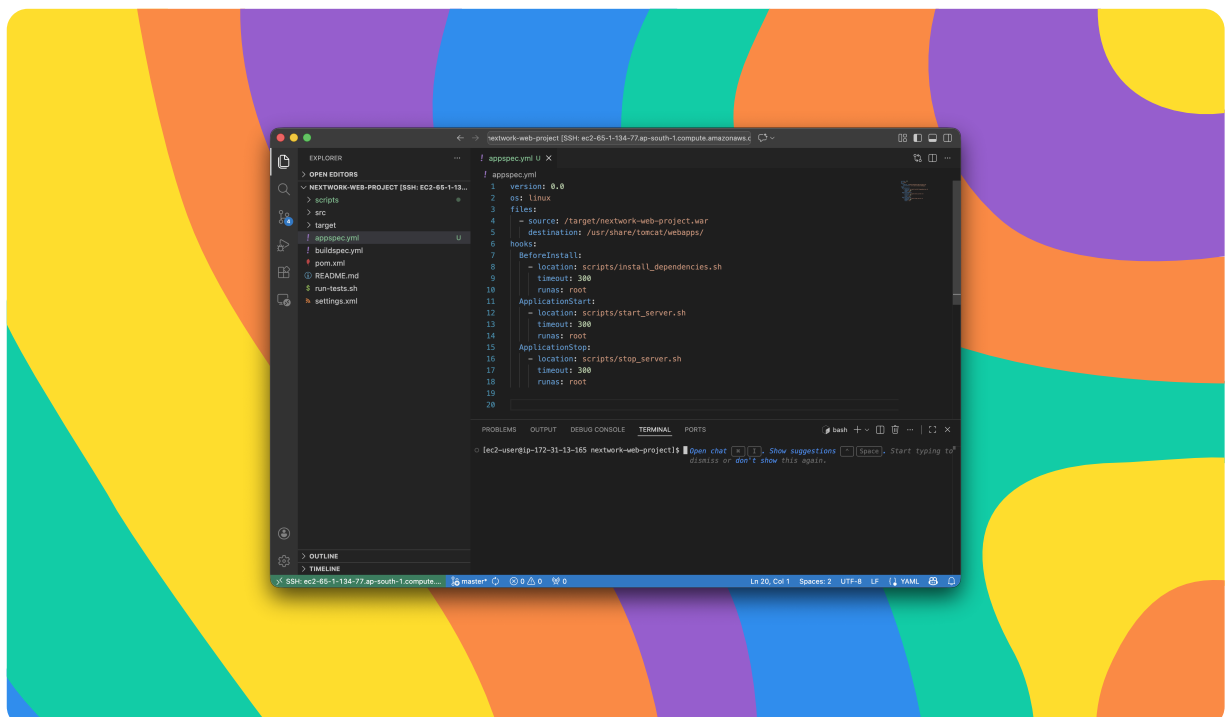
The `start_server.sh` script starts the Tomcat and Apache services required to run the Java web application. It also enables both services so they automatically restart whenever the EC2 instance reboots. This ensures the application becomes available immediately after deployment without manual intervention.

The `stop_server.sh` script checks whether Apache (httpd) and Tomcat processes are currently running on the server. If they are active, it stops the corresponding services using `systemctl` to avoid errors during deployment. This ensures the application services are safely shut down before updating or replacing deployment files.

appspec.yml

appspec.yml is a configuration file that tells AWS CodeDeploy how to deploy the application to the target server. It defines which files should be copied from the build artifact to specific locations on the EC2 instance and which scripts should run during different deployment stages. This ensures the deployment follows a structured and automated process every time.

The buildspec.yml file was updated by modifying the artifacts section to include the deployment configuration and scripts along with the .war file. This ensured that appspec.yml and the entire scripts/ folder are packaged into the build output produced by CodeBuild. By doing this, the build artifact contains everything AWS CodeDeploy needs to execute a complete deployment.





Setting Up CodeDeploy

A CodeDeploy application is the main container that represents the software being deployed and helps organize all related deployment resources. A deployment group is a set of deployment rules and target EC2 instances where that application will be installed and managed. In short, the application defines what is being deployed, while the deployment group defines where and how it gets deployed.

CodeDeploy needs an IAM role so it can securely perform deployment actions on AWS resources without using manual credentials. This role allows CodeDeploy to access EC2 instances, retrieve build artifacts from S3, and manage deployment-related services like Auto Scaling and load balancers. Assigning a role ensures permissions are controlled through least-privilege access while enabling automated deployments.

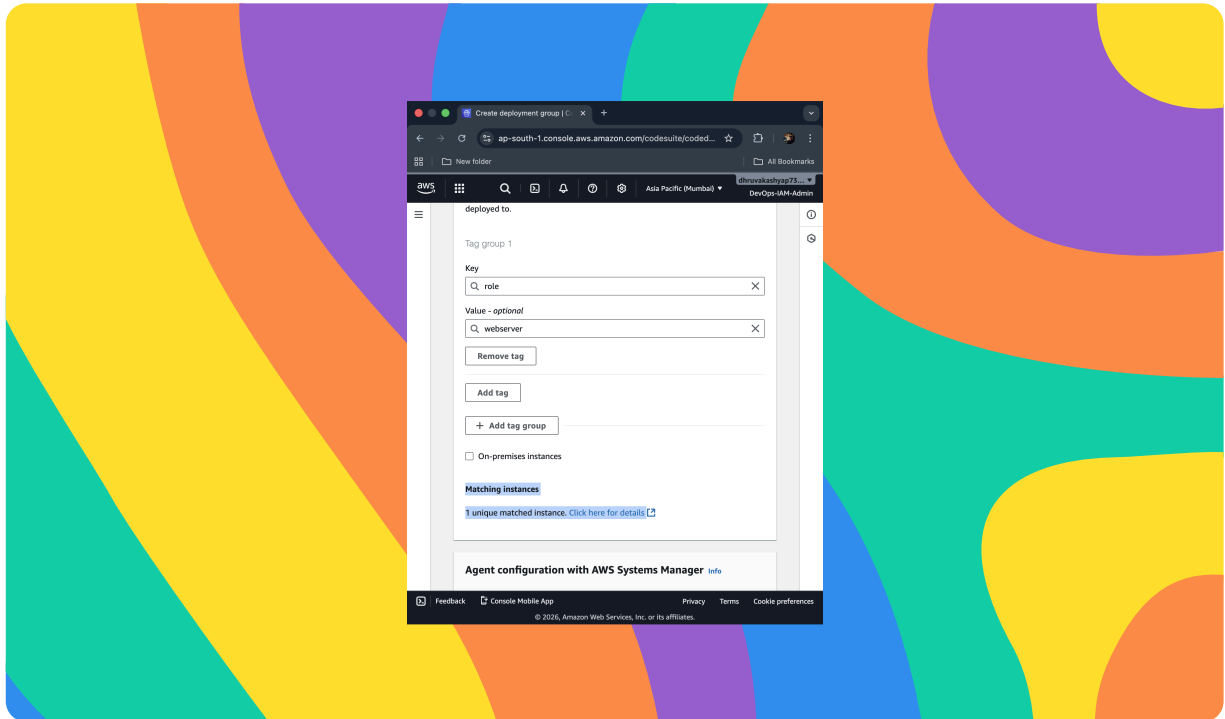
Tags are used for EC2 instance targeting so AWS CodeDeploy can automatically identify which instances should receive the deployment. By matching the tag key and value (such as `role: webserver`), CodeDeploy selects the correct deployment server without needing a fixed instance ID. This makes deployments scalable and easier to manage, because any new instance with the same tag can be included automatically.



Dhruva Kashyap

NextWork Student

nextwork.org

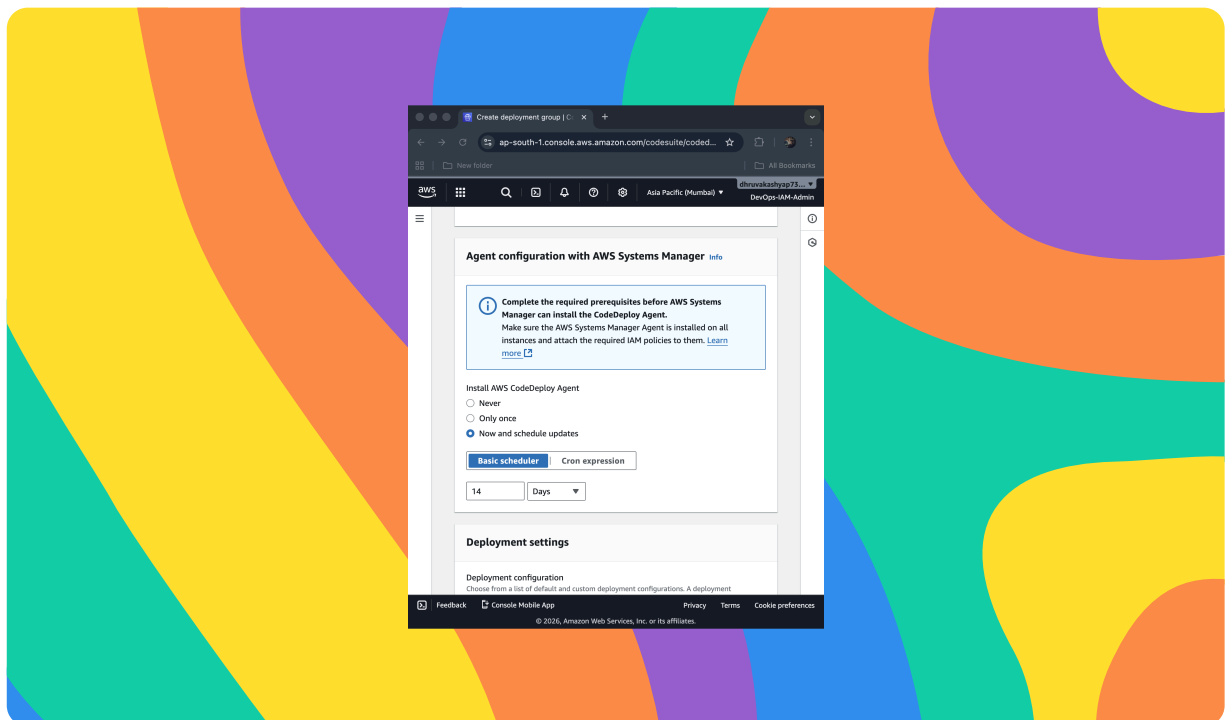




Deployment configurations

CodeDeploy deployment configuration options control how updates are rolled out across the instances in a deployment group. Common options include **AllAtOnce** for deploying to all instances simultaneously, **OneAtATime** for updating instances one by one, and **HalfAtATime** for deploying to 50% of instances first and then the rest. These strategies help balance deployment speed, risk, and service availability during releases.

The CodeDeploy Agent is a lightweight software installed on the target EC2 instance that enables communication between the instance and AWS CodeDeploy. It receives deployment instructions, downloads the application revision, and runs the lifecycle scripts defined in `appspec.yml`. This agent is required so CodeDeploy can automate deployments directly on the server.





Success!

A CodeDeploy deployment is a single execution of delivering a specific application revision to the target instances in a deployment group. It includes a unique deployment ID and tracks the full history of the update process. During a deployment, CodeDeploy manages the required steps such as stopping services, copying files, running scripts, and starting the application.

A revision location is the storage path where CodeDeploy retrieves the application build artifact for deployment. It points to the exact file that contains the deployable package, such as a .zip bundle stored in an Amazon S3 bucket. This ensures CodeDeploy installs the correct version of the application onto the target EC2 instances.

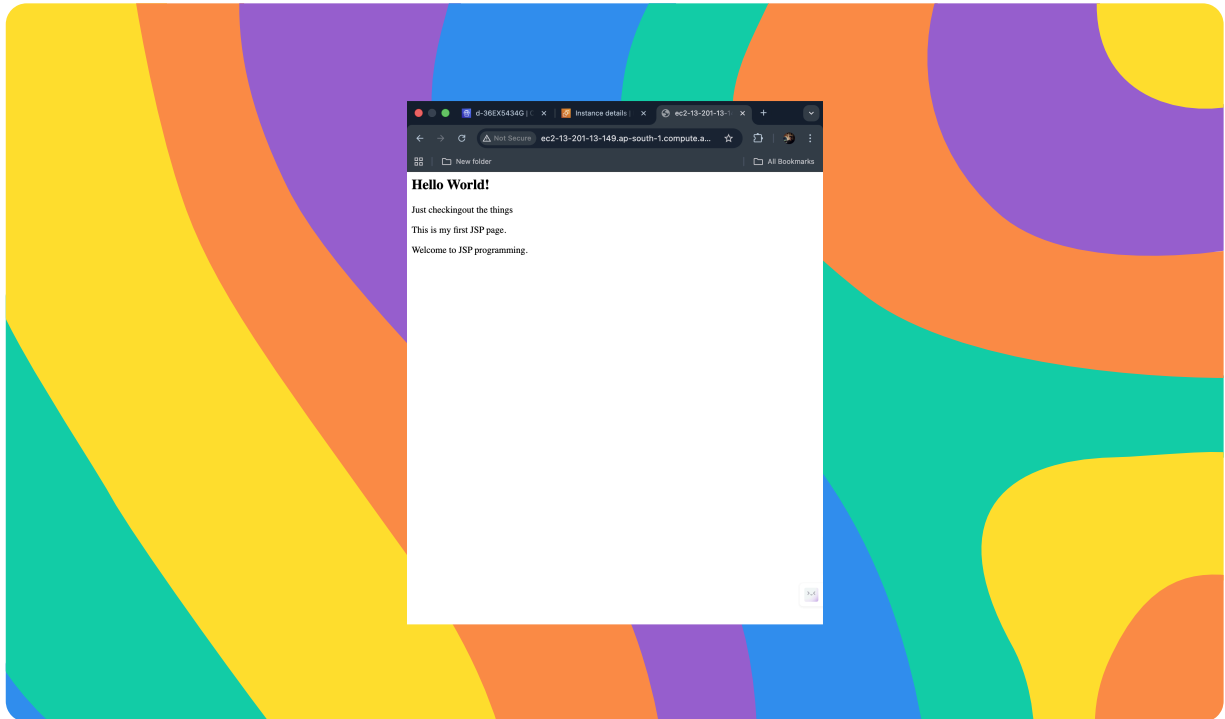
I verified the CodeDeploy success by confirming the deployment status changed to Success and that all deployment lifecycle events completed without errors. I then opened the deployment EC2 instance's Public IPv4 DNS in a browser using HTTP to access the application. Seeing the web app load correctly in the browser confirmed the deployment was completed successfully.



Dhruva Kashyap

NextWork Student

nextwork.org





Disaster Recovery

The error introduced was an intentional typo in the deployment script by using `systemctl` instead of the correct `systemctl` command in `stop_server.sh`. This invalid command causes the script to fail immediately and return a non-zero exit code during the CodeDeploy lifecycle event. As a result, the deployment fails on purpose to demonstrate how rollback and recovery work.

Rollbacks were enabled to automatically recover the application if the new deployment failed during execution. This helps minimize downtime by attempting to restore a working version instead of leaving the system in a broken state. Enabling rollbacks adds a safety layer that makes deployments more reliable and less risky.

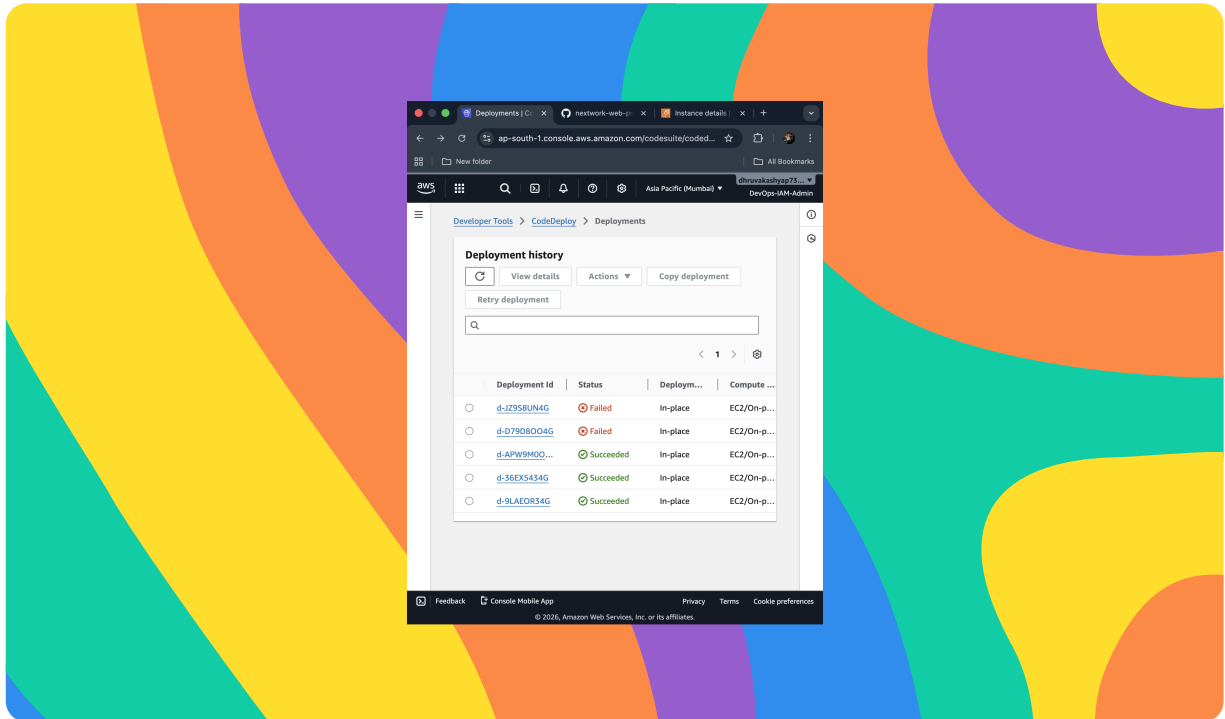
In a production environment, automatic rollbacks can be implemented by integrating AWS CodePipeline with CodeBuild and CodeDeploy, so the pipeline tracks each revision and deploys a specific build artifact version. The deployment stage can be configured to trigger a rollback action when a failure is detected, allowing traffic to return to the last known stable revision automatically. This approach reduces downtime by ensuring recovery happens immediately without manual intervention.



Dhruva Kashyap

NextWork Student

nextwork.org





nextwork.org

The place to learn & showcase your skills

Check out nextwork.org for more projects

